v0.2.0 PRE-RELEASE

COMPREHENSIVE USER GUIDE

RDAT

Translation Copilot

Professional English-to-Arabic Computer-Assisted Translation environment with a three-tier predictive ghost-text pipeline powered by local Ollama LLMs.

مساعد الترجمة الاحترافي من الإنجليزية إلى العربية

ENGINE	PLATFORMS	LICENSE
Ollama + Gemma 4	Win / Mac / Linux	MIT

Dr. Waleed Mandour / waleedmandour.org

github.com/waleedmandour/rdat

PART 1

Introduction

RDAT: Translation Copilot is an AI-powered translation workspace purpose-built for professional English-to-Arabic translation workflows. It combines a segmented translation editor with a three-tier predictive ghost-text pipeline that delivers real-time suggestions while keeping translators in full control of every word.

The system is designed around the principle that professional translators should never have to choose between speed and quality. Suggestions appear inline as you type, drawn from progressively smarter tiers that balance latency against nuance: from instant corpus matches to RAG-enriched LLM completions that respect your loaded terminology, to cloud-scale Gemini translations for complex or ambiguous passages.

As of v0.2.0, RDAT ships in two complementary forms, sharing a single React frontend. The Tauri desktop app (Windows, macOS, Linux) uses Ollama as the primary local LLM engine. The PWA on Vercel runs in the browser with optional WebLLM via WebGPU. Both paths share the same editor, glossary, and tier pipeline.

PART 2

Install Ollama and Pull Your First Model

RDAT uses Ollama as its primary local LLM engine. Ollama runs entirely on your machine, so no data leaves your device. Follow these steps to get your first model running.

1. **Download and install Ollama**

   Visit `ollama.com/download` and download the installer for your platform. Run the installer. On Windows and macOS, Ollama starts automatically. On Linux, run `ollama serve` in a terminal.

2. **Launch RDAT: Translation Copilot**

   Open the RDAT desktop app. On first run, an onboarding modal will appear to verify Ollama is running. Click **Check now**. If successful, the modal dismisses automatically. If you skip it, the app runs in degraded LTE-only mode.

3. **Pull Gemma 4 E2B (recommended starter)**

   Open the **Models** panel from the sidebar. Click **Pull** on `gemma4:e2b`. The 1.5 GB download takes 2 to 10 minutes depending on your connection. A progress bar shows live status. Larger alternatives: `gemma4:e4b` (3 GB, higher quality) or `qwen3:4b` (2.5 GB, balanced).

4. **Load the model**

   After the pull completes, click **Load** on the same model card. The status banner at the top turns green and shows `Active model: gemma4:e2b`. The model stays loaded for 5 minutes of inactivity (Ollama's default keep-alive).

Tip: If you skip Ollama installation, the app still works, but only the LTE (corpus match) tier will produce suggestions. To enable the local LLM tier, install Ollama and load a model.

PART 3

Translate with Ghost-Text Suggestions

Click **Launch Editor** from the dashboard. Paste English source text in the left panel. It is auto-segmented into sentences. Click any segment in the right panel to start translating. As you type Arabic, ghost-text suggestions appear inline with a colored badge showing the source tier.

BADGE	SOURCE TIER	LATENCY
LTE	Local Translation Engine: instant corpus match using your glossary and the built-in seed corpus (approximately 80 EN-AR pairs).	under 5 ms
GEMMA4	Local LLM via Ollama: RAG-augmented, glossary-aware. This is the PRIMARY engine of the project.	200 to 2000 ms
GEMINI	Cloud fallback via Gemini 2.5 Flash. Optional, requires your own API key.	500 to 3000 ms

The pipeline consults each tier in order. The first tier to return a confident result supplies the ghost-text. If a higher-latency tier is already computing, its result replaces the current suggestion upon arrival, enabling progressive refinement without blocking your typing flow.

PART 4

Keyboard Shortcuts

`Tab` Accept full suggestion	`Ctrl + Right` Accept next word	`Alt + ]` Cycle candidates	`Esc` Dismiss suggestion
`Ctrl + Enter` Confirm segment	`Alt + ]` Cycle alternatives	`Click` Focus segment	`Volume icon` Pronounce translation

PART 5

The AI Models Panel

The Models panel is the control center for your local LLM engine. It shows the active adapter (Ollama in desktop mode, WebLLM in browser mode), lists available models, and lets you pull, load, and remove them.

Status Banner: Green when connected, amber when not running.

Model Catalog: Gemma 4 E2B, E4B, Qwen 3, Llama 4.

Pull Button: Downloads a model from Ollama registry.

Load Button: Selects a pulled model as active.

Remove Button: Deletes a model from local storage.

Recheck Button: Re-runs Ollama detection if it was just started.

PART 6

Troubleshooting: Ollama Not Detected

If the app shows "No local engine available" even though Ollama is installed, the new diagnostics panel will show you exactly what was tried and why it failed. Here is how to read it and fix common issues.

```
[x] http://127.0.0.1:11434/api/tags (Connection refused)
[x] http://localhost:11434/api/tags (Connection refused)

[v] http://127.0.0.1:11434/api/tags (Success)
```

Connection refused: Ollama is not running. Open the Ollama app from your Start Menu (Windows) or Applications folder (macOS), or run `ollama serve` in a terminal (Linux).

Timeout: Ollama is running but slow to respond. Click **Recheck** after a few seconds. The app also auto-retries once after 3 seconds on initial launch to handle cold-start races.

Corporate proxy / VPN: If you have HTTP_PROXY set or a corporate VPN, the app bypasses proxies for loopback traffic automatically. No action needed.

Diagnostic screenshots: If you report an issue, include a screenshot of the diagnostics box. It shows exactly which URLs were tried and which errors occurred, making remote diagnosis possible.

PART 7

Optional: Gemini Cloud Fallback

The Gemini cloud tier is optional. It provides a fallback when local tiers yield low confidence or when no local model is available. To enable it:

1. **Get a Gemini API key**

   Visit `aistudio.google.com/apikey` and create a free key. Note: after June 19, 2026, unrestricted standard keys require restriction (HTTP referrer or IP). The Rust-side proxy in the desktop app handles this better than direct browser calls.

2 **Enter and test the key**

Open the **API Keys** panel. Paste your key. Click **Test**. If the key is valid, you will see a green checkmark with a test translation. The key is stored in your browser's localStorage only, never on a server.

PART 8

Glossary and Terminology Management

The Glossary panel lets you import, browse, and manage terminology databases. Loaded glossary entries serve two purposes: they are indexed into the LTE for instant matching, and they provide RAG context for the local LLM tier, so your translations respect domain-specific vocabulary.

JSON Import: Load custom glossary files in JSON format.

Seed Corpus: 80 pre-loaded EN-AR pairs covering CAT, tech, academic, legal.

IndexedDB Storage: All data persists locally across sessions.

RAG Integration: Top 5 relevant entries injected into LLM prompts.

PART 9

Engine Settings

Three engine modes are available via the Models panel settings section:

MODE	TIERS ACTIVE	BEST FOR
Hybrid	LTE + Local LLM + Gemini	Maximum quality with graceful fallback (recommended)
Local	LTE + Local LLM only	Zero data egress, full offline, privacy-sensitive
Cloud	Gemini API only	Complex passages, or when no local model is available

PART 10

Contact and Support

RDAT: Translation Copilot is developed by Dr. Waleed Mandour, Assistant Lecturer at Sultan Qaboos University, Oman. For questions, feedback, or collaboration, please use the contact details below.

Dr. Waleed Mandour

Assistant Lecturer, Sultan Qaboos University, Oman

Email: w.abumandour@squ.edu.om

Website: waleedmandour.org

ORCID: 0000-0002-9262-5993

GitHub: github.com/waleedmandour/rdat

PART 11

Citation

If you use this software in academic work, please cite it using the DOI below.

APA CITATION (7TH EDITION)

Mandour, W. (2026). *RDAT: Translation Copilot (Version 0.2.0)* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.21256765

PART 12

Acknowledgements

AI-Assisted Development

The development of RDAT: Translation Copilot has been performed with the assistance of multi-AI agents. The primary AI assistant used throughout the development lifecycle was **GLM 5.2** (2026) by Z.ai, which was used for code generation, architectural analysis, debugging, documentation, and visual design. Additional AI agents were consulted for code review, research synthesis, and cross-platform testing guidance. These AI-assisted development tools significantly accelerated the project's iterative refinement across the PWA and Tauri desktop distribution channels.

Open Source Dependencies

RDAT: Translation Copilot builds upon the following open-source projects: **Tauri 2** (desktop application framework), **React 19** and **Vite 6** (frontend), **Ollama** (local LLM runtime), **@mlc-ai/web-llm** (browser LLM inference), **@google/genai** (Gemini SDK), **Tailwind CSS 4** (styling), **Zustand 5** (state management), and **Motion** (animations). Full license information is available in the repository.